Software Requirements Specification

for

Motion Detection Survelliance System using Raspberry PI camera module

Version 4.0.0

Prepared by Mikołaj Żyszkiewicz

Lodz University of Technology

20.05.2026

Table of Contents

Revision History

Name	Date	Reason for Changes	Version
Mikołaj Żyszkiewicz	20.05.2026	Initial version	1.0.0
Mikołaj Żyszkiewicz	02.06.2026	Updated the product features and added a UML graph	2.0.0
Mikołaj Żyszkiewicz	08.06.2026	Updated the System Features, Extrenal Interface Requirements and create a Class graph.	3.0.0
Mikołaj Żyszkiewicz	15.06.2026	Updated the rest of the SRS File, added a State Diagram and an Activity Diagram	4.0.0

1. Introduction

1.1. Purpose

This document specifies the complete software requirements for the Motion Detection Surveillance System (MDSS) firmware, Version 4.0.0, developed by Mikołaj Żyszkiewicz at Lodz University of Technology. The described firmware is intended to run on a Raspberry Pi single-board computer equipped with a compatible camera module and shall provide autonomous, edge-computed video surveillance without reliance on external cloud infrastructure or continuous human oversight. This specification covers all functional and non-functional requirements necessary for the design, implementation, testing, and maintenance of the system. It shall serve as the authoritative reference document for all development and verification activities associated with this product.

1.2. Document Conventions

This document follows the IEEE Std 830-1998 structure. **SHALL** denotes a mandatory requirement, **SHOULD** a recommendation, and **MAY** an optional behavior. Functional requirements are numbered **REQ-X.Y** and non-functional requirements **NF-X.Y**, where *X* identifies the feature group and *Y* the sequential number within it. All timestamps use the ISO 8601 format (YYYYMMDD_HHMMSS).

1.3. Intended Audience and Reading Suggestions

This document is intended for the following reader types:
Embedded Developers and Software Engineers form the primary audience. They should read the document in full, with particular focus on Sections 3 (System Features), 4 (External Interface Requirements), 5 (Functional Requirements), and 6 (Non-Functional Requirements), as these sections define the precise behavior the implemented firmware shall exhibit.

Testers and QA Engineers should focus on Sections 5 and 6, where individually verifiable requirements are stated. The state diagram in Section 3.7 and the activity diagram in Section 5.1.2 provide additional visual context useful for test case derivation.

Academic Supervisors and Project Evaluators at Lodz University of Technology are advised to begin with Sections 1 and 2 for a product scope and architectural overview, and then proceed to Section 3 for a concise summary of system capabilities.

System Administrators responsible for deployment and operation of the device should primarily consult Section 4.1 (User Interfaces Overview) and Section 2.6 (User Documentation). All readers are encouraged to begin with Sections 1 and 2 before proceeding to the sections most relevant to their role.

1.4. Product Scope ----

The software described herein will be used as the firmware for the Raspberry Pi based Motion Detection Surveillance System. The software is designed for autonomous video monitoring of an area in real-time, detecting threats in either interior or restricted areas with no need for

ongoing human supervision or reliance upon remote cloud computing infrastructure. The firmware communicates with the camera module installed on the Raspberry Pi to perform environmental telemetry processing and event logging (capture) of digitally recorded evidence only when there are significant visual anomalies. The software performs all computer vision and data management tasks at the network edge to minimize notification time (latency), optimize the use of local storage space, and reduce energy consumption. Therefore, the software offers a secure and cost-effective alternative to commercially available security networks, consistent with the design principles of embedded systems paradigm for stand-alone, edge-computed IoT deployments.

1.5. References

[1] IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications, IEEE, 1998.

[2] Raspberry Pi Foundation, Raspberry Pi Camera Module v2 / v3 Documentation, available at: https://www.raspberrypi.com/documentation/accessories/camera.html

[3] Raspberry Pi Ltd, The Picamera2 Library, 2023, available at: https://datasheets.raspberrypi.com/camera/picamera2-manual.pdf

[4] Itseez / OpenCV Community, OpenCV 4.x Python Documentation, available at: https://docs.opencv.org/4.x/

[5] Python Software Foundation, smtplib — SMTP Protocol Client, Python 3 Standard Library Documentation, available at: https://docs.python.org/3/library/smtplib.html

[6] Pallets Projects, Flask Documentation v3.x, available at: https://flask.palletsprojects.com/

[7] Python Cryptographic Authority, cryptography — Python Cryptography Library, available at: https://cryptography.io/en/latest/

[8] Raspberry Pi Foundation, Raspberry Pi OS Documentation, available at: https://www.raspberrypi.com/documentation/computers/os.html

2. Overall Description

2.1. Product Perspective

The Motion Detection Surveillance System specified in this document is a new, self-contained firmware application engineered to operate entirely at the network edge. It does not serve as a direct follow-on member of an existing product family, nor is it a legacy modification of a pre-existing software suite. Instead, it was conceived as a standalone solution to address distinct architectural vulnerabilities common in contemporary commercial security infrastructures.

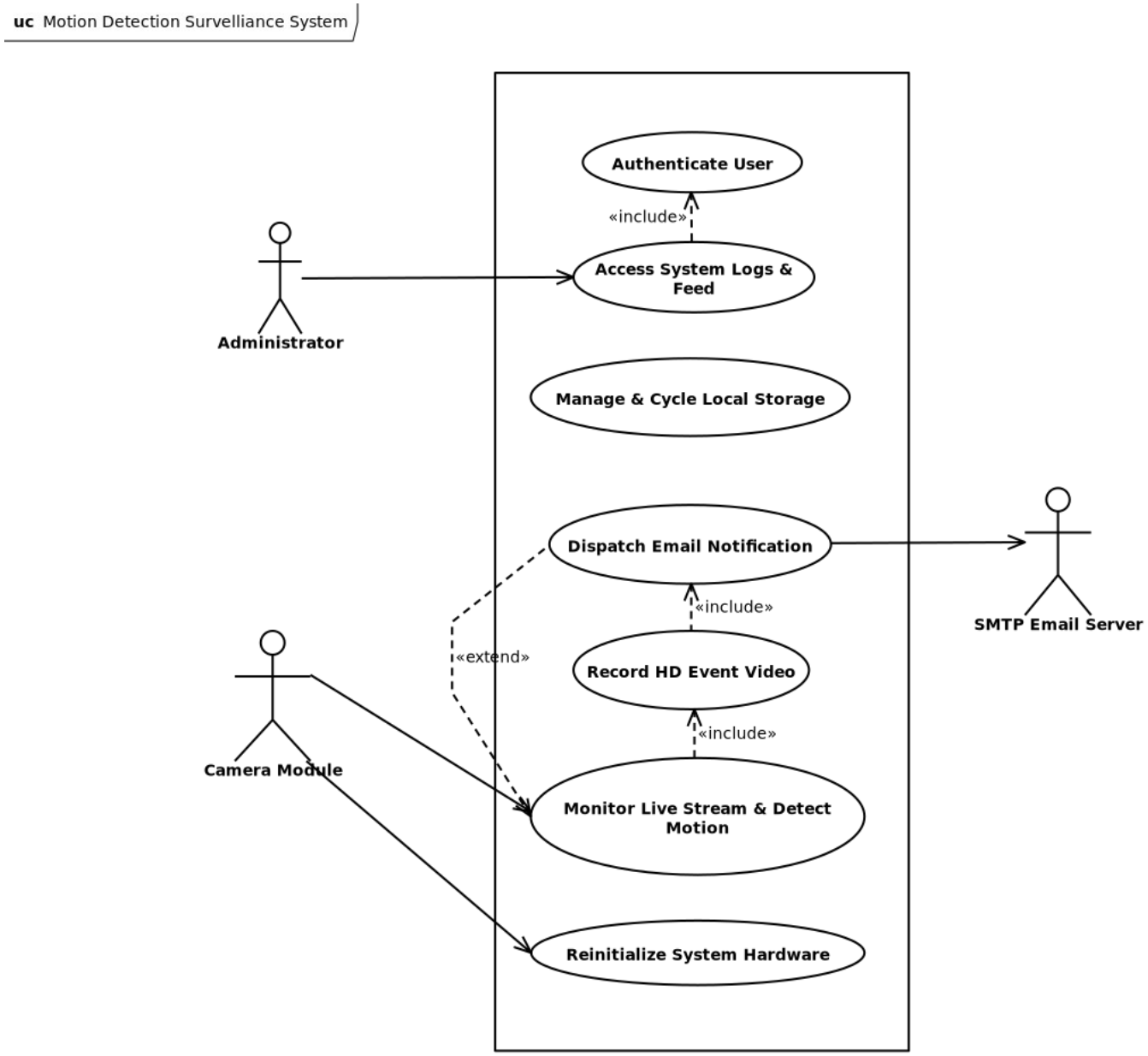
Historically, localized surveillance has depended heavily on continuous human oversight or high-bandwidth cloud video recording (CVR) pipelines. These dependencies introduce single points of failure, including high operational subscription costs, severe latency bottlenecks, and significant data privacy risks. This product originates from the engineering need to decouple real-time visual threat detection from external networks. By shifting the computational workload to a low-cost, edge-computed hardware platform—specifically the Raspberry Pi

architecture—the system serves as a decentralized replacement for traditional, cloud-dependent IP camera systems.

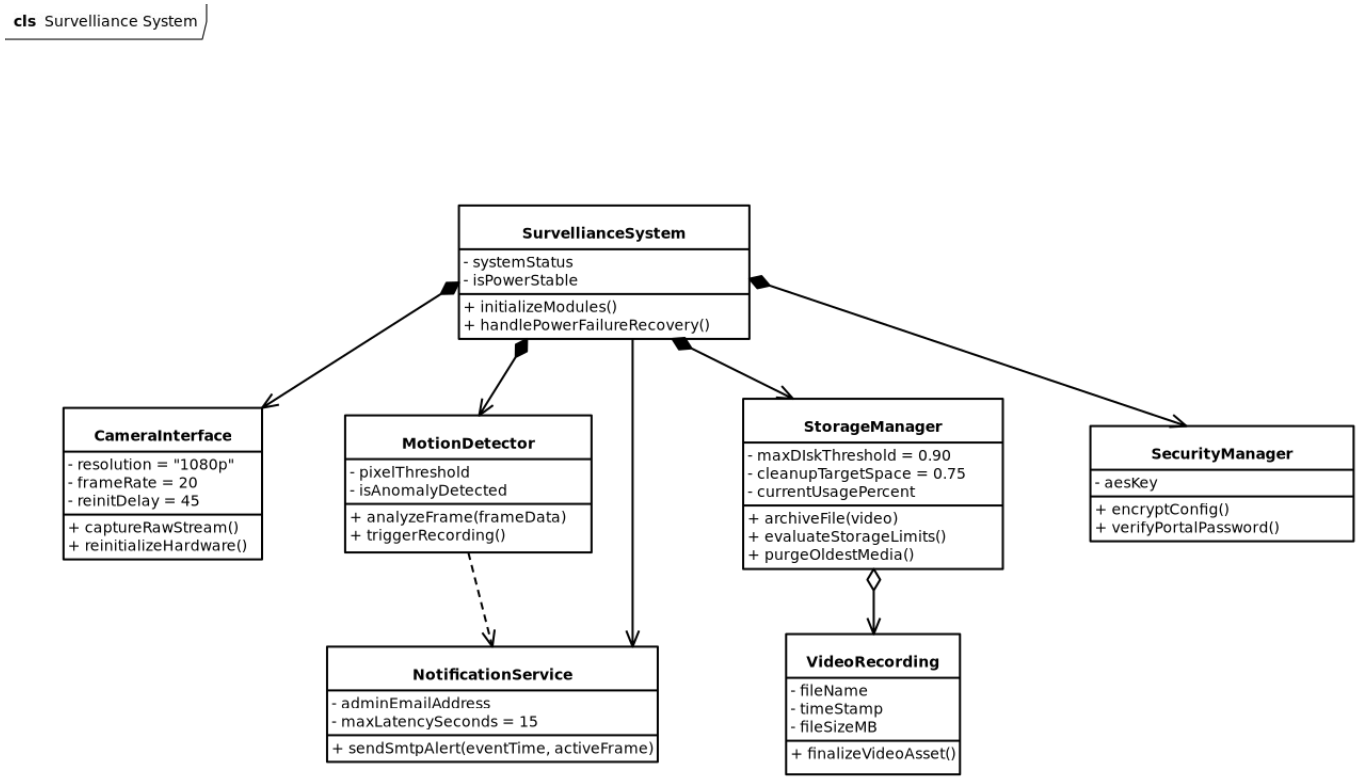
As a self-contained entity, the firmware manages the entire lifecycle of a security event locally. It directly controls the hardware abstraction layer of the camera module, processes the video stream frame-by-frame, and manages its own localized storage and alerting protocols. This independent design ensures that the system remains fully operational even during network isolation, establishing a secure, privacy-focused foundation for localized Internet-of-Things (IoT) security deployments.

2.2. Product Features

- **Real-Time Motion Detection:** Continuously processes the raw video stream frame-by-frame at the edge to detect localized visual anomalies.
- **Automated Video Recording:** Initiates high-definition local recording (1080p at 20fps) for a fixed window whenever motion thresholds are crossed.
- **Decentralized Media Management:** Automatically formats, labels, and archives video files locally, with built-in rolling storage cleanup when disk usage peaks.
- **Instant Alert & Telemetry Dispatch:** Generates and distributes secure email alerts complete with precise timestamps and evidence images directly to an administrator.
- **Edge Data Security & Authentication:** Restricts system configuration access via hashed credentials and encrypts sensitive network keys locally.
- **Fault-Tolerant Power Recovery:** Self-heals and automatically reinitializes critical hardware components following electrical power disruptions.



The internal firmware architecture of the Raspberry Pi Motion Detection Surveillance System is strictly object-oriented to ensure modularity and ease of maintenance. The system utilizes dedicated software interfaces to standardize edge device orchestration, while strictly separating the volatile, real-time frame processing and computer vision logic from the non-volatile local disk storage and rolling media management mechanics. The UML Class Diagram below illustrates the composition of the main surveillance controller, its tight encapsulation of peripheral hardware modules, and its operational media asset aggregation methods.



2.3. User Classes and Characteristics

This product defines a single user class:
System Administrator (Primary and Sole User). The system administrator is the only human operator who interacts with the surveillance firmware. This individual is responsible for initial hardware assembly, firmware deployment, credential configuration, and ongoing monitoring of system-generated alerts. The administrator is assumed to possess intermediate technical knowledge — sufficient to configure a network-connected device, manage an email account, and operate a standard web browser — but is not expected to have expertise in embedded software development or computer vision. All interaction with the system occurs remotely through the administrator web portal described in Section 4.1, supplemented by automated email notifications dispatched by the firmware itself. The administrator holds the highest and only privilege level in the system; no guest, read-only viewer, or unprivileged user class is defined in this version of the product.

2.4. Operating Environment

The firmware shall operate in the following environment:
Hardware Platform: The primary compute unit shall be a Raspberry Pi 4 Model B with a minimum of 2 GB RAM. The camera peripheral shall be a Raspberry Pi Camera Module v2 (Sony IMX219, 8 MP) or Camera Module v3 (Sony IMX708, 12 MP), connected via the Raspberry Pi's MIPI CSI-2 interface. Local non-volatile storage shall be provided by a MicroSD card of at least 32 GB capacity (Class 10 / UHS-I rating or faster), or alternatively a USB-connected external drive. Network connectivity shall be provided by the onboard IEEE 802.11b/g/n/ac Wi-Fi adapter (2.4 / 5 GHz) or the onboard 100/1000BASE-T Ethernet port.

Operating System: The firmware shall run on Raspberry Pi OS Lite (64-bit, Debian Bookworm-based, kernel 6.x) in headless mode, with no graphical desktop environment installed.

Runtime Environment: Python 3.10 or higher is required. The following third-party libraries must be available: picamera2 (≥ 0.3), OpenCV/cv2 (≥ 4.5), Flask (≥ 2.0), and cryptography (≥ 3.0). The following Python standard library modules are also used: smtplib, hashlib, email, threading, and os.

Power Supply: The system shall be powered by a stable 5 V / 3 A USB-C power supply under normal operating conditions. System behavior following unplanned power interruptions is defined in Section 3.5 and REQ-4.1.

2.5. Design and Implementation Constraints

The following constraints shall govern the design and implementation of the firmware:
Language Constraint: The firmware shall be implemented entirely in Python 3.10 or higher, consistent with the hardware abstraction libraries available on Raspberry Pi OS.

Target Hardware: The firmware shall target the Raspberry Pi hardware platform exclusively. Portability to generic x86/x64 or other ARM platforms is out of scope for this version.

Edge-Only Architecture: The firmware shall not require or depend on any external cloud storage endpoint, remote processing service, or third-party API beyond a standard SMTP mail relay. All computer vision processing, media storage, and data management shall occur locally on the device.

No RTOS: The system shall not use a real-time operating system. Timing constraints shall be addressed through optimized multi-threaded frame-processing loop design on top of Raspberry Pi OS.

Memory Budget: The firmware shall operate within the memory envelope of a Raspberry Pi 4 with 2 GB RAM, accounting for concurrent frame-processing, web server, email dispatch, and storage management threads running simultaneously.

Cryptographic Standards: All sensitive configuration data stored at rest shall be encrypted using AES-256-GCM. Password hashing shall use SHA-256. No deprecated or weak cryptographic algorithms shall be used.

Network Dependency for Alerts: Email alert dispatch depends on network availability. The firmware shall not block or stall the recording pipeline when the network is temporarily unavailable; delivery failures shall be logged locally and the surveillance pipeline shall continue uninterrupted.

Single Administrator Account: The web portal shall support exactly one administrator account. Multi-user management and role-based access control are out of scope for this version.

2.6. User Documentation

The following documentation components shall be delivered alongside the firmware:
README.md — A plain-text installation and quick-start guide covering hardware assembly, Raspberry Pi OS preparation, Python dependency installation, initial credential setup, and system launch instructions. This document shall be maintained in the project's version control repository.

Configuration Reference — A document listing all configurable system parameters (motion detection threshold, recording duration, storage cleanup thresholds, SMTP server settings, administrator email address), their valid value ranges, default values, and encryption behavior.

Administrator Portal Guide — A brief guide describing how to access the web portal over the local network, authenticate, interpret the motion event log, view the live camera stream, and modify system configuration settings.

No graphical help system, interactive tutorials, or printed manuals are within scope for this release. All documentation shall be authored in Markdown format and stored alongside the firmware source code in the project repository.

2.7. Assumptions and Dependencies

Assumptions:

It is assumed that the system administrator has access to a functioning SMTP relay or email provider (for example, Gmail with an app-specific password, or a local mail server) and can supply the necessary credentials during initial system configuration. It is further assumed that the Raspberry Pi deployment environment has a stable, mains-powered electrical supply under normal operating conditions; power interruptions are treated as exceptional, recoverable events as specified in Section 3.5. The monitored area is assumed to provide sufficient ambient illumination for the camera sensor to produce usable frames; low-light or night-vision operation is not guaranteed unless the specific camera module variant in use includes infrared capability. The local network is assumed to provide DHCP-assigned IP addressing; if a static IP address is required for reliable portal access, the administrator is responsible for configuring this at the OS level prior to deployment. Finally, it is assumed that the administrator's email client correctly renders MIME multipart messages with embedded JPEG attachments without filtering or quarantining them.

Dependencies:

The firmware depends on the **picamera2** library (libcamera backend) for all camera hardware interaction; a breaking change to its Python API would require corresponding firmware updates. Frame differencing, contour analysis, and JPEG encoding depend on **OpenCV (cv2, ≥ 4.5)**. The administrator web portal depends on **Flask (≥ 2.0)** for HTTP routing and server-side session management. AES-256-GCM encryption of configuration data at rest depends on the **Python cryptography library (≥ 3.0)**. Finally, the firmware depends on V4L2 and CSI-2 kernel modules available specifically within Raspberry Pi OS; portability to other Linux distributions is not guaranteed without modification.

3. System Features

This section provides a brief overview of the primary functional capabilities of the Motion Detection Surveillance System firmware. Each feature is described concisely here; detailed functional requirements, stimulus/response sequences, and implementation constraints are elaborated in Section 5.

3.1. Real-Time Motion Detection Engine

The system shall continuously analyze the live video stream on a per-frame basis and classify a motion event whenever the pixel-level difference between two consecutive frames meets or exceeds the configured detection threshold. This capability forms the core trigger mechanism for all downstream recording and alerting operations.

3.2. Automated Video Capture and Local Archiving

Upon a confirmed motion trigger, the system shall initiate a fixed-duration high-definition video recording and persist the resulting file to local non-volatile storage under a timestamp-based filename. The system shall operate this pipeline entirely at the edge, without dependency on any external network storage or cloud infrastructure.

3.3. Alert and Telemetry Dispatch

The system shall generate and transmit an email notification to the pre-configured administrator address upon completion of each recording. The notification shall include the event timestamp and an embedded JPEG image corresponding to the most visually active frame captured during the motion event.

3.4. Rolling Media Lifecycle Management

The system shall monitor local disk utilization on a continuous basis. When storage consumption exceeds a defined upper threshold, the system shall autonomously purge the oldest archived media files in chronological order until utilization falls below the defined lower threshold, ensuring uninterrupted recording capability without administrator intervention.

3.5. Fault-Tolerant Power Recovery

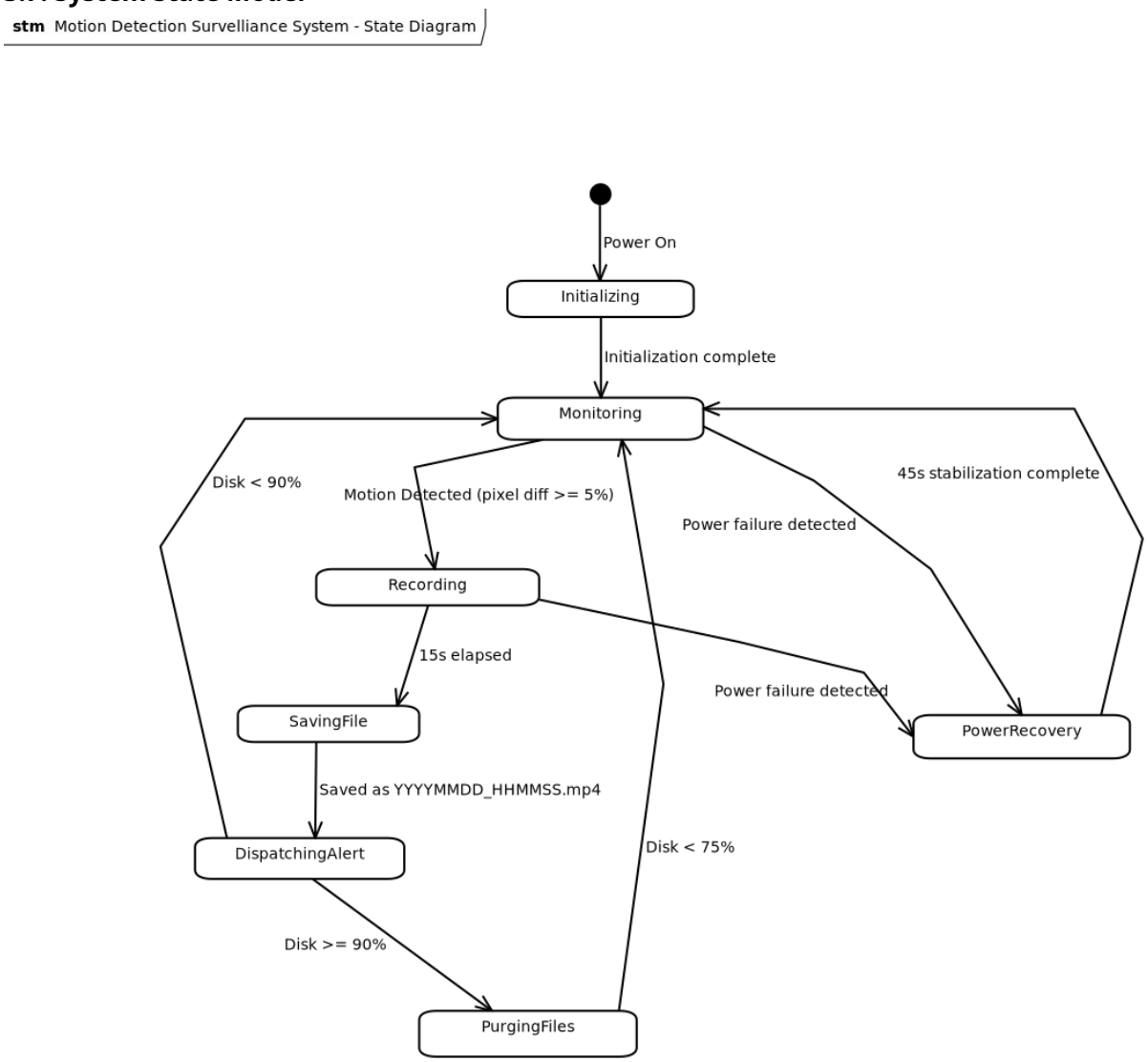
The system shall detect loss and subsequent restoration of electrical power and shall execute a structured hardware reinitialization sequence following a fixed stabilization delay, ensuring all peripheral components have reached a stable operating state before surveillance operations resume.

3.6. Secure Configuration and Access Control

The system shall enforce authentication for all administrative access and shall store sensitive configuration parameters — including SMTP credentials, Wi-Fi keys, and administrator contact

details — in encrypted form in non-volatile memory. No plaintext credentials shall be written to disk at any point during system operation.

3.7. System State Model



4. External Interface Requirements

4.1. User Interfaces Overview

The Motion Detection Surveillance System exposes a minimal, administrator-only web portal as its primary user-facing interface. Given the embedded, headless nature of the Raspberry Pi deployment, there is no physical display or local GUI. All human interaction occurs remotely via a browser-based interface served over the local network by the firmware's built-in web server.

The web portal provides the following high-level functions:

- **Authentication screen:** A login form requiring a username and SHA-256-validated password before granting access. All other portal views are inaccessible without a valid authenticated session.
- **Live stream view:** Displays the real-time MJPEG or HLS video feed from the camera module, allowing remote visual monitoring of the surveilled area.
- **Event log view:** Presents a paginated, timestamped list of past motion events, each linked to its associated recorded video file and alert image.
- **System configuration panel:** Allows the administrator to modify SMTP credentials, recipient email address, motion detection sensitivity, and recording duration parameters. Input fields for sensitive values (passwords, Wi-Fi keys) shall be masked. All configuration changes must be confirmed before being persisted to encrypted storage.

The interface shall function correctly on any standards-compliant modern browser (Chrome, Firefox, Edge, Safari) without requiring additional plugins. Error states (e.g., camera unavailable, disk full warning) shall be surfaced via clearly visible status indicators on the portal dashboard.

4.2. Hardware Interfaces

- **Camera Module (Raspberry Pi Camera Module v2 / v3 or compatible)**
  The firmware interfaces with the camera module through the Raspberry Pi's MIPI CSI-2 serial interface. Logically, the software communicates with the device via the picamera2 Python library, which abstracts the low-level V4L2 kernel driver. The firmware configures the camera to operate in continuous preview mode, capturing raw frames at a resolution sufficient for 1080p output at 20fps. The firmware also monitors the hardware readiness state of the camera module during the post-power-failure recovery sequence (45-second stabilization window) before reinitializing the capture pipeline.
- **Local Storage (MicroSD Card / USB Drive)**
  All video recordings and configuration data are persisted to the local block storage device mounted on the Raspberry Pi. The firmware interacts with the storage through the host operating system's standard POSIX file I/O interface. The system monitors available disk space and triggers the rolling cleanup routine (Section 3.4) autonomously.
- **Network Interface (Wi-Fi / Ethernet)**
  The firmware uses the Raspberry Pi's onboard network interface (IEEE 802.11 Wi-Fi or 100/1000BASE-T Ethernet) to transmit alert emails via SMTP and to serve the administrator web portal over HTTP/HTTPS. Wi-Fi credentials are stored in encrypted form and loaded by the firmware at initialization.

4.3. Software Interfaces

Operating System — Raspberry Pi OS (Linux, Debian-based)
The firmware runs as a user-space Python application on top of Raspberry Pi OS Lite. It relies on the OS for process scheduling, file system access, network stack, and hardware abstraction. No RTOS is used; real-time constraints are met through optimized frame-processing loops and threading.

Camera Driver — `picamera2` **(libcamera backend)**
The firmware imports `picamera2` as its primary hardware abstraction interface to the camera module. This library wraps the `libcamera` framework and exposes a Python API for frame capture, resolution control, and hardware lifecycle management.

Computer Vision — OpenCV (`cv2`**)**
Incoming frames are processed using OpenCV for pixel-level frame differencing and motion threshold evaluation. OpenCV also handles JPEG encoding of alert images. The firmware uses OpenCV's frame subtraction and contour analysis routines to classify events.

Email Dispatch — `smtplib` **/** `email` **(Python Standard Library)**
Alert emails are composed and transmitted via Python's built-in `smtplib` interface using SMTP with TLS (port 587 or 465). The email payload is assembled using the `email.mime` module, with the JPEG alert image attached as a MIME multipart body.

Web Server — Flask (or lightweight equivalent)
The administrator web portal is served by an embedded Flask application running on the Raspberry Pi. Flask handles routing for the login, live stream, event log, and configuration endpoints. Session management and authentication state are handled server-side.

Cryptographic Storage — `cryptography` **library (AES-256) /** `hashlib` **(SHA-256)**
Sensitive configuration parameters are encrypted at rest using the `cryptography` Python library with AES-256-GCM. Web portal password validation is performed using SHA-256 digests via Python's built-in `hashlib` module. No plaintext credentials are stored on disk at any time.

5. System Features/Modules

5.1. Motion Detection and Video Recording Pipeline

5.1.1 Description and Priority

This feature constitutes the core operational loop of the entire surveillance system and is classified as **Critical** priority. The firmware shall continuously capture live frames from the camera module and evaluate each incoming frame against the immediately preceding one for pixel-level differences using OpenCV frame subtraction. When the computed difference meets or exceeds the configured motion threshold, the system shall confirm a motion event and immediately initiate a fixed-duration, high-definition video recording. Upon completion of the recording window, the resulting video file shall be persisted to local non-volatile storage under a timestamp-based filename. This feature acts as the primary trigger mechanism for all downstream operations — alert dispatch, storage management, and event logging — defined in Sections 5.2 through 5.5. All other system features depend on the successful and continuous execution of this pipeline.

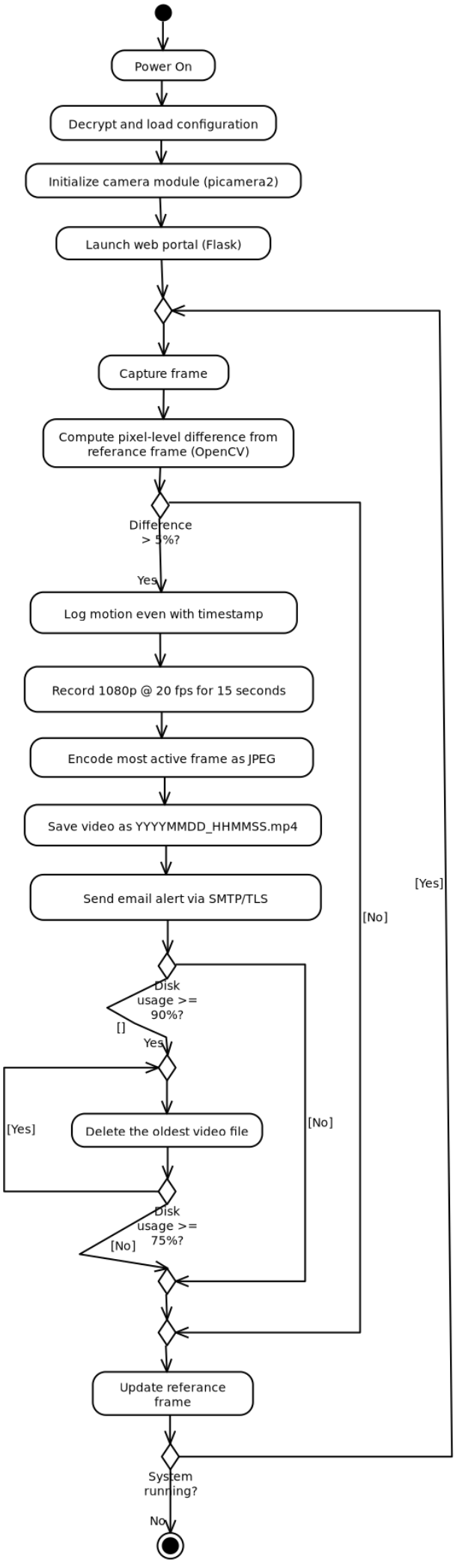
5.1.2 Stimulus/Response Sequences

Stimulus: The system powers on and completes initialization successfully.
Response: The system shall begin continuous frame capture from the camera module and enter the motion detection loop immediately.
Stimulus: A captured frame differs from the preceding reference frame by less than 5% at the pixel level.
Response: The system shall discard the frame comparison result, update the stored reference frame, and proceed to capture the next frame. No recording shall be initiated.

Stimulus: A captured frame differs from the preceding reference frame by 5% or more at the pixel level.
Response: The system shall log the confirmed motion event with the current timestamp and shall immediately initiate a 1080p / 20 fps video recording session lasting exactly 15 seconds.

Stimulus: The 15-second recording window elapses.
Response: The system shall finalize the recording, identify the most visually active frame from the captured clip, and persist the video file to local non-volatile storage using the filename format YYYYMMDD_HHMMSS.mp4.

act Motion Surveillance System - Activity Diagram



5.1.3 Functional Requirements

REQ-1.1: Whenever the System detects a 5% difference in pixels between two consecutive frames, the System should start recording a 1080p video with 20fps for a period of 15 seconds.

REQ-1.2: Once the 15 second recording time is completed, the System should save the recorded video in the local folder using the format YYYYMMDD_HHMMSS.mp4.

REQ-1.3: After successfully saving a new video, the System should automatically send an e-mail notification containing a timestamp of the captured motion event along with a JPEG image of the most active frame to the configured administrator address within 10 seconds.

REQ-1.4: Once the local storage usage exceeds 90%, the System should remove oldest media files sequentially until the disk space usage drops below 75%. REQ-1.5: Once a system power failure occurs, the System will reinitialize the camera module interface once it becomes ready after 45 seconds.

5.2 — Alert and Telemetry Dispatch

5.2.1 Description and Priority

Following each successful video recording event, the system shall automatically compose and transmit an email alert to the pre-configured administrator address. The notification shall serve as the primary real-time situational awareness mechanism, delivering a precise event timestamp and a JPEG still image of the most visually active frame from the recorded clip. This feature is classified as **High** priority. The practical usefulness of the system as a security tool depends directly on the timeliness and reliability of these alerts.

5.2.2 Stimulus/Response Sequences

Stimulus: A video file has been successfully written to local storage following a confirmed motion event.
Response: The system shall identify the most visually active frame from the completed recording, encode it as a JPEG image, compose an email containing the motion event timestamp and the JPEG as a MIME multipart attachment, and transmit the message to the configured administrator address via SMTP/TLS within 10 seconds of the file save completing.

Stimulus: The SMTP connection attempt fails due to network unavailability or misconfiguration.
Response: The system shall log the failed delivery attempt — including the timestamp and reason for failure — to the local event log. The system shall not block or retry indefinitely; the recording pipeline shall resume immediately.

5.2.3 Functional Requirements

REQ-2.1: Upon successful completion of each motion-triggered video recording, the system SHALL transmit an email notification to the pre-configured administrator address within 10 seconds of the file being saved. The notification SHALL include the UTC timestamp of the motion event and a JPEG image of the most visually active frame from the recording, delivered as a MIME multipart attachment.

REQ-2.2: All outbound email communication SHALL use SMTP with TLS (port 587 or 465). Plaintext SMTP connections SHALL NOT be used.

REQ-2.3: If the SMTP connection cannot be established, the system SHALL log the failure to the local event log and SHALL continue recording pipeline operations without blocking or waiting for delivery confirmation.

5.3 Rolling Storage Management

5.3.1 Description and Priority

The system operates on a storage medium of finite capacity and shall remain capable of archiving new recording events indefinitely, without requiring administrator intervention. To achieve this, the firmware shall continuously monitor local disk utilization and shall autonomously purge the oldest archived media files whenever storage consumption crosses the defined upper threshold. This feature is classified as **High** priority; storage exhaustion would prevent the archiving of new events and render the recording pipeline non-functional.

5.3.2 Stimulus/Response Sequences

Stimulus: Local disk utilization is evaluated following a successful video archive operation and is found to be below 90%.
Response: No cleanup action shall be taken. The system shall resume normal monitoring operations immediately.

Stimulus: Local disk utilization is found to be at or above 90% following a storage check.
Response: The system shall begin purging archived media files in strict chronological order, starting from the oldest file, until disk utilization falls below 75%. Each deleted file shall be recorded in the local event log.

5.3.3 Functional Requirements

REQ-3.1: The system SHALL evaluate local disk utilization after each video recording is successfully archived to non-volatile storage.

REQ-3.2: When disk utilization reaches or exceeds 90%, the system SHALL autonomously and permanently delete archived video files in chronological order — oldest first — until disk utilization falls below 75%.

REQ-3.3: Each file deletion performed by the storage management routine SHALL be recorded in the local event log, including the filename and the disk utilization percentage at the time of deletion.

6. Nonfunctional Requirements ----

6.1. Performance

NF-1.1: The motion detection engine should process incoming raw video stream coming from the Raspberry Pi

NF-1.2: The end to end latency from the moment physical motion was detected until its final delivery to the configured administrator address in an email should not exceed 15 seconds.

6.2. Security

NF-2.1: Configuration payloads such as SMTP passwords, Wi-Fi keys, and administrator email parameters shall be securely stored in non-volatile memory using AES-256 encryption.

NF-2.2: User authentication using SHA-256 hashing algorithm for password validation shall be mandatory for the web portal through which system logs and live streaming feeds are accessed.

6.3. Reliability

NF-3.1: 99.5% uptime shall be achieved by the system during any test period of 30 days except the time taken for site-wide electrical infrastructure maintenance.